VARA HF Modem: Cross-Platform Rust Reimplementation

Lab notes and evidence ledger

Engineering research log

30 October 2026

Abstract

This log records the initial setup and reconnaissance for an independently implemented Rust modem intended to interoperate with VARA HF and be installable on macOS, Windows, and Linux. It distinguishes direct observations from hypotheses. No modem protocol, on-air framing, modulation, or DSP behavior has yet been implemented or established. The current Rust crate is a scaffold, not a functioning modem.

Contents

1 Project objective and boundaries **3**
1.1 Objective . . . 3
1.2 Research boundaries . . . 3

2 Chronicle **3**

3 Reference materials and provenance **8**
3.1 Supplied VARA package . . . 8
3.2 Documentation age . . . 9

4 Ghidra setup and analysis ledger **9**
4.1 Setup . . . 9
4.2 Imported programs . . . 9
4.3 Evidence ledger . . . 9
4.4 Saved Recon bookmarks . . . 11

5 Rust scaffold and cross-platform environment **11**

6 Workspace organization **12**

7 Implementation roadmap (proposed) **12**
7.1 Planning targets and architecture . . . 12
7.2 Phases and exit gates . . . 13
7.3 Cross-cutting standards and acceptance . . . 14
7.4 Initial execution order . . . 14

8 Signal data acquisition and corpus (proposed) **14**
8.1 Purpose and evidence policy . . . 14

8.2 Candidate data sources 15
8.3 Recording preservation and metadata 15
8.4 Coverage and capture campaigns 16
8.5 Storage, analysis, and corpus admission 17
8.6 Initial acquisition sprint and gate 17

9 Current readiness **18**

10 Chronicle maintenance **18**

11 Next work **18**

12 References **19**

1 Project objective and boundaries

1.1 Objective

Develop an independent, cross-platform Rust modem that can interoperate with VARA HF. Intended initial operating-system families are macOS, Windows, and Linux, with ARM64 and x86_64 build targets for each. Hardware and native audio validation will be required; compile checks on one host do not demonstrate device-level compatibility.

1.2 Research boundaries

- Use the supplied reference application and controlled observations to infer externally observable behavior. Keep the Rust implementation independent; do not copy decompiled implementation code or proprietary assets into the Rust crate.
- Keep confirmed facts, hypotheses, and unresolved questions separate. A UI string or function cross-reference is a lead, not proof of protocol semantics.
- Review applicable licenses and law before analysis or redistribution.
- Limit dynamic transmission tests to audio loopback or a properly shielded/conducted test setup; avoid unintended over-the-air transmissions.

2 Chronicle

Date	Work recorded
2026-10-30	Reviewed the product description and local resources. Established the goal as an independent Rust reimplementation with macOS, Windows, and Linux support. The vendor description identifies computer sound-card operation, adaptive link management, ARQ, and a TCP command interface as product-level features; these descriptions are not a detailed protocol specification.
2026-10-30	Installed Ghidra 12.1.4 alongside the existing 12.2 user profile because the provided GhidraMCP 7.0.0 extension declares Ghidra 12.1.4 compatibility. Installed the extension in the 12.1.4 profile, used Java 21, and registered the workspace bridge in `.vscode/mcp.json`. The MCP health endpoint reported version 7.0.0 at port 8089.
2026-10-30	Set up the VARA source artifacts and Ghidra analysis project. Imported the installer and then the extracted application executable. Added Recon bookmarks to the application analysis and saved the Ghidra project.
2026-10-30	Installed UTM 5.0.8 and CrystalFetch 2.2.0 to prepare a Windows 11 ARM64 reference VM on an Apple Silicon Mac. The user reported on 2026-10-30 that the Windows 11 ISO was still downloading. At the last available workspace check, no ISO was visible in the accessible Downloads directory. The Documents directory was inaccessible to the tool session, and UTM CLI automation returned macOS Apple Events permission error -1743. VM creation and reference execution remain pending.

Date	Work recorded (continued)
2026-10-30	Installed Rust stable 1.99.1, Cargo, rustfmt, Clippy, and rust-analyzer. Added target standard libraries for macOS ARM64/x86_64, Windows ARM64/x86_64, and Linux ARM64/x86_64. All six targets passed a `cargo check` of the initial dependency-free crate.
2026-10-30	Created the Rust scaffold in `rust/`, with separate API, audio, portable modem, and platform modules. Formatting, Clippy, tests, and CLI `-help`/`-version` smoke checks passed. There are currently zero implemented tests and no modem behavior.
2026-10-30	Organized supplied reference materials under `reference/vara/` and `reference/ghidra/`, preserving the active Ghidra project, `.venv/`, and VS Code MCP configuration at their existing paths. Added workspace overview documentation. The workspace is not a Git repository, so no Git status/diff is available.
2026-10-30	Created this LaTeX lab log and a Beamer progress deck. Recorded the verified setup, reference fingerprints, initial static-analysis clues, Rust scaffold checks, and pending Windows VM work. Going forward, update the chronicle as work proceeds, noting material actions, evidence, validation results, and blockers before concluding each work session.
2026-10-31	Located the downloaded Windows installer in Downloads: `26100.4349.250607-1500.ge_release_svc_refresh_CLIENTCONSUMER_RET_A64FRE_en-us.iso`. The file is a bootable ISO 9660 image, 5,267,005,440 bytes. Its filename identifies an AArch64 build, but the Windows image architecture has not yet been verified from its install image. Local SHA-256: `347624272ec7e209de37ff5a5ff49190a616eec792e176fd4ba19fd78f278e2a`. This checksum was measured locally; it has not been compared against a publisher-provided checksum. UTM 5.0.8 is installed and open, but `utmctl` still receives macOS Apple Events permission error -1743. No UTM virtual machine has been created.
2026-10-31	Located an open-source UTM MCP server and registered it alongside Ghidra in `.vscode/mcp.json`. The published `mcp-utm==0.1.2` package failed to start because its dependency resolver installed MCP SDK 2.x while its code imports the removed `mcp.server.fastmcp` API. Switched to the MIT-licensed `neverprepared/mcp-utm` source at pinned commit `5f6bab58061f995b90cd5a5af4101211b6768fd4`, constrained the MCP SDK to version 1.x, and verified a real stdio handshake, enumeration of all 22 registered tools, and a successful `list_vms` call. The inventory is empty; no VM was created. The server exposes state-changing tools, including permanent VM deletion; no such action was invoked.
2026-10-31	The user created a UTM VM named `Windows`. Via the UTM MCP, verified that it is registered with the QEMU backend, has 8,192 MiB memory, and has a 5,031 MiB removable drive attached (consistent in size with the downloaded Windows ISO). Started the VM and confirmed its status is `started`. The MCP configuration reports `cpu_cores=0`; this may be an unsupported/unknown QEMU configuration value and must be checked in UTM before relying on a CPU allocation. Windows setup has not yet been observed or completed.

Date	Work recorded (continued)
2026-10-31	The user reports Windows installation is complete, the guest desktop was reached, and ARM64 architecture was confirmed. This is user-reported status; the MCP does not provide direct guest-screen evidence. The VM inventory and configuration were re-queried: `Windows` is registered as QEMU and running, with 8,192 MiB memory and three drives, including a removable 5,031 MiB drive. The MCP still reports `cpu_cores=0`, which should be checked in UTM before running the reference modem. Next: verify guest readiness and CPU allocation, then install and test VARA HF v4.9.0.
2026-10-31	The user checked UTM and reports that the VM is configured with 10 virtual processors. This resolves the CPU-allocation check; the earlier MCP value of `cpu_cores=0` was not an accurate indication of the configured count. The VM is ready for the next gate: install and test VARA HF v4.9.0.
2026-10-31	Before guest installation, reviewed the extracted package's `License.rtf`. It describes VARA HF as shareware and includes a term prohibiting reverse compilation, modification, translation, and disassembly. That restriction conflicts with this project's VARA reverse-engineering goal. No guest installation or dynamic behavioral testing was performed during this license review.
2026-10-31	The user states that they have written authorization covering the entire VARA reverse-engineering and testing scope. This is recorded as a user-reported authorization; the authorization document was not reviewed or retained. Work may proceed within the user's stated scope. The UTM VM has no shared folders, and the available UTM MCP does not expose a guest shell or guest UI-control tool. Prepared a separate 4.4 MiB ISO containing only the provided VARA HF v4.9.0 setup executable for transfer into the guest. The original installer remains unchanged. Attaching the ISO requires a stopped VM; the user authorized a graceful stop after saving guest work.
2026-10-31	Gracefully stopped and restarted the Windows VM with user approval. The installer ISO was validated locally, but UTM MCP's `attach_drive` operation failed with AppleScript error -10006; the VM was not left stopped. The installed `utmctl` supports guest file push/exec, but the Windows guest reports that the QEMU guest agent is not installed/running. Verified the UTM shared-network host address is 192.168.64.1. Continue with a temporary host-local transfer of the single setup executable; the user must complete Windows installer UI and license prompts because no guest UI-control channel is available.
2026-10-31	Staged the original VARA HF v4.9.0 setup executable in a temporary directory and exposed only that file through a temporary HTTP endpoint bound to the UTM private interface at 192.168.64.1:8765. A host-side download smoke test matched the source SHA-256 `b7d67fd9fcc2043a49e58ae8202fc699bdb6520255d21aefd258260c767e04bd`. No installer was run in the guest. The endpoint is temporary and will be stopped after the user confirms the download.

Date	Work recorded (continued)
2026-10-31	The user reports that VARA is installed in the Windows VM and identifies `~/Downloads/utm-share/` as the UTM shared folder. The UTM MCP now confirms that directory is registered for the running VM. Its `app/` payload was compared recursively with the extracted v4.9.0 reference payload; no differences were found. The user's installation report has not yet been independently verified inside the guest, and no modem runtime test has been performed. Stopped the temporary HTTP transfer server after this update; the shared folder is now the intended host/guest data path.
2026-10-31	Continued static command-interface reconnaissance in the existing Ghidra program. The UI string `TCPCommandPort(default:8300)` at `00448260` has a data reference in `FUN_004c3780`; invalid command/KISS TCP port messages reference `FUN_004c61c0`. The strings `CQFRAME`, `LISTENCQ`, `LISTENON/OFF`, `DRIVELEVEL`, `VERSION`, `CONNECTED`, `DISCONNECTED`, `BUSYON/OFF`, and `BITRATE` are referenced by code including `FUN_004cdf80` and nearby event handlers. A UI description says the KISS interface is for APRS applications and should be disabled when running Winlink. These are string/reference observations only; they do not establish command framing, direction, responses, or behavior. Decompilation attempts for the implicated functions again returned `cancelled`; assembly context confirms string constants are used at those code locations but is not sufficient to specify the protocol. Runtime validation remains pending. The UTM VM is running and the shared directory is registered, but its guest IP was not discoverable from the host's current ARP table and the QEMU guest agent is unavailable. No packets or commands were sent to the guest.
2026-10-31	The user reports that Windows has no network device. UTM MCP reports the VM network mode as `shared`; however, no guest IPv4 address is available from `utmctl` and the host ARP table has no guest entry. UTM's Windows guest guide says SPICE guest tools include the network drivers and that networking may not work until they are installed. Recommended next step is to shut down the guest cleanly, confirm networking is enabled in the VM's QEMU settings with Shared Network mode, install UTM's Windows Guest Tools from its CD menu, reboot, and check Windows Device Manager and `ipconfig`. UTM/libslirp may not support ping, so lack of ping replies alone is not a failure signal.
2026-10-31	Downloaded the official UTM Guest Tools ISO from https://getutm.app/downloads/utm-guest-tools-latest.iso to `~/Downloads/utm-share/utm-guest-tools-latest.iso`, the user's UTM shared folder. The 121 MiB image is ISO 9660 and contains `utm-guest-tools-0.1.271.exe` and the VirtIO Windows driver license text. SHA-256: `65b6a69b392ee01dd314c10f3dad9ebbf9c4160be43f5f0dd6bb715944d9095b`. UTM's official Windows guest-tools guide states Windows ARM64 is supported. This is the appropriate UTM/QEMU guest driver bundle; VMware VMXNET3 is not the adapter model configured for this VM. The ISO has not yet been mounted in the VM or installed.

Date	Work recorded (continued)
2026-10-31	The user clarifies that Windows reboots when the VM starts with either VirtIO PCI or Intel E1000 configured; no stop code or other error is reported. The UTM process itself is not reported as crashing. UTM still reports the VM as started, but no guest IPv4 address is discoverable. Next diagnostic step: fully stop the VM, disable/remove the emulated network device temporarily, and test whether Windows boots stably. If it does, install UTM Guest Tools from the attached Guest Tools CD while networking is disabled, then re-enable a UTM-compatible adapter and retest. Change only one variable at a time; do not reconfigure CPU or additional devices during this isolation check.
2026-10-31	The user asks whether Wine could be used as an alternate VARA research environment. Assessment: feasible as a compatibility experiment, but not a replacement for Windows validation. The imported executable is 32-bit x86 and the application package contains VB6 ActiveX controls (including Winsock and serial-control components); audio and COM behavior may differ under Wine. No Wine environment has been installed or run. Before proceeding, choose a host/guest architecture that can run this 32-bit application and preserve the authorized scope; compare only non-transmitting behavior first.
2026-10-31	The user proposes a Wine environment inside an x86 Linux VM. This is a reasonable fallback to investigate while the Windows guest has startup reboots: an x86_64 Linux guest can run under QEMU emulation on the Apple Silicon host, and Linux typically has built-in VirtIO networking. Expect lower VM performance than ARM64 virtualization, and treat VARA's VB6 ActiveX/Winsock/audio/serial behavior under Wine as compatibility-specific. No ISO has been downloaded and no Linux VM has been created. Proposed setup: Debian amd64 guest, snapshot before installing Wine, install 32-bit Wine support and required VB6/COM dependencies, then test VARA idle and non- transmitting before attempting any audio work.
2026-10-31	The user selected Fedora for the x86 Linux Wine VM. Downloaded Fedora Workstation 43 Live x86_64 from the official Fedora release directory into `~/Downloads/utm-share/Fedora-Workstation-Live-43-1.6.x86_64.iso`. The bootable ISO is 2,742,190,080 bytes. Its SHA-256 `2a4a16c009244eb5ab2198700eb04103793b62407e8596f30a3e0cc8ac294d77` matches the hash in Fedora's published CHECKSUM file. The CHECKSUM signature was not independently verified. The ISO and CHECKSUM file are now in the shared folder. No Fedora VM has been created; UTM should use Emulate with x86_64 architecture (not ARM virtualization).
2026-10-31	Checked host QEMU availability before attempting the requested QEMU/KVM installation. This Apple Silicon Mac already has Homebrew QEMU 11.1.1 installed. `qemu-system-x86_64 -accel help` lists only TCG; the ARM QEMU binary lists HVF and TCG. KVM is a Linux-kernel accelerator and is not available on macOS. No package installation or upgrade was performed. An x86_64 Fedora guest can run under TCG emulation but will be substantially slower; native ARM64 guests can use HVF.

Date	Work recorded (continued)
2026-10-31	Downloaded Fedora Everything netinst x86_64 43-1.6 to the shared folder. Its SHA-256 `f4d06a40ce4fb4a84705e1a1f01ca1328f95a2120d422ba5f719af5df62d0099` matches Fedora's published CHECKSUM entry; the manifest signature was not independently verified. Added `tools/fedora-qemu-install.sh`, which creates a separate 64 GiB QCOW2 disk and starts a Kickstart-based Workstation install under QEMU TCG. The existing UTM `Linux` VM is left untouched. The automated install was attempted, but QEMU exited without producing a usable Fedora installation. The QCOW2 image is structurally valid with a 64 GiB virtual size but only 196 KiB allocated; no Fedora first boot was verified. The script refuses to overwrite this disk.
2026-10-31	Continued read-only Ghidra analysis without a Windows runtime. Cross-references map `CQFRAME`, `LISTEN ON/OFF`, and `VERSION` to `FUN\_004cdf80`; the listen strings also reference `FUN\_004d45a0`. Status strings reference separate candidates: `CONNECTED` to `FUN\_004d14e0`, `DISCONNECTED` to `FUN\_004d1ac0`, `BUSY ON/OFF` to `FUN\_004d2640`/ `FUN\_004d27a0`, `BITRATE` to `FUN\_004d2da0`, and `VERSION VARA` to `FUN\_004d33b0`. Decompilation still returns `cancelled`; these are string-reference leads, not verified command handlers or semantics. Both UTM guests remain stopped.
2026-11-01	Drafted an implementation roadmap from the supplied proposal. The phases, schedule, crate boundaries, and candidate waveform parameters are planning assumptions, not measured VARA behavior or completed work. The roadmap keeps protocol and DSP implementation gated on reference evidence and adds the project's existing macOS target to the proposal's Windows/Linux scope.
2026-11-01	Captured a proposed signal-data acquisition guide in this log. It prioritizes authentic VARA HF recordings, immutable originals, provenance, sidecar metadata, hashes, evidence labels, and validated-corpus gates. Its suggested tools, formats, and capture campaigns are recommendations; no new recordings or signal measurements were made by documenting the guide.

3 Reference materials and provenance

3.1 Supplied VARA package

The local HF package is the VARA HF v4.9.0 setup ZIP in `reference/vara/installers/`. It contains `VARA setup (Run as Administrator).exe`. The setup executable was identified as a 32-bit Windows PE GUI installer. Its strings identify Inno Setup setup data 5.1.2. The installer payload was extracted using `innoextract` to `reference/vara/extracted-v4.9.0/app/`.

The extracted `VARA.exe` is a PE32 x86 GUI application. Ghidra identified language `x86:LE:32:default`, compiler `windows`, image base `00400000`, 808 functions, and 9,868 symbols. The on-disk executable size is 1,310,720 bytes. These are tool observations about this imported image, not claims about the architecture of unseen components.

Artifact	Size (bytes)	SHA-256
VARA.exe	1,310,720	c69b8c81f888032a67f3c15fe57ef5cc dabef6eba5be148a5a4d7eb44f00486f
VARAHF500.dat	734,771	4be7fd1a8f1ee2df4c529e8d4c34205f a06a26b82fbceb02abde7295c049ded3
VARAHF2300.dat	720,716	2c823ce719944dccf90194129b7c95f9 1b9add7c33a46bb0c80b8401d0edcfe5
VARAHF2750.dat	872,206	e3e516f70f7710dc7992196573ce0141 f5604d978e2701dced26eb06de3aede6
Data.dat	65,000	58590d0ba95f819f5bccca97291043db 96405b8304b244121c702bec6d994c23

The three VARAHF*.dat files are distinct payloads whose names include 500, 2300, and 2750. Their internal roles and formats have not been confirmed. Do not treat filenames or entropy measurements as a DSP specification.

3.2 Documentation age

The extracted application bundle contains a quick guide marked VARA HF 4.7; it is older than the v4.9.0 executable and must be treated as potentially stale. The v4.9.0 ChangeLog and executable strings are the more version-specific local leads. The product website is useful for high-level context, not implementation detail.

4 Ghidra setup and analysis ledger

4.1 Setup

Ghidra 12.1.4 and extension GhidraMCP 7.0.0 are installed locally. The workspace-level MCP configuration points to the local Python bridge executable. The plugin health check returned status ok, server kind gui, version 7.0.0. The project shows 205 plugin endpoints; default tool groups were available.

The active project is represented by vara.gpr and vara.rep/. The workspace copy of vara.gpr is zero bytes in the filesystem snapshot, while the repository directory contains project state and indexed data. Preserve both together and validate the project through Ghidra rather than inferring validity from the stub file alone.

4.2 Imported programs

- Installer loader: VARA setup (Run as Administrator).exe, imported under Ghidra project folder /recon; Ghidra detected x86 32-bit Delphi and 456 functions.
- Application: extracted VARA.exe, imported under project folder /reference/VARA_HF_4.9.0; analysis reported 808 functions.

4.3 Evidence ledger

Class	Observation	Interpretation / limit
Verified	The GUI text includes `TCP Command Port (default: 8300)` at `00448260`; it also includes TCP port range validation text.	Confirms a displayed command-port setting/default in this build. Does not establish command grammar, line framing, default runtime bind address, or which socket serves every application function.
Static clue	The bundled VARA HF 4.7 quick guide describes application communication on ports 8300 and 8301 by default.	Older guide; exact port roles and v4.9 behavior require runtime confirmation.
Verified	Strings show `waveIn*`, `waveOut*`, and `winmm.dll` including open, buffer, start/stop, and write operations.	Strong evidence that the Windows application references WinMM wave audio APIs. It motivates a portable Rust audio abstraction; it does not determine the replacement backend or complete audio format.
Verified	UI text includes separate input and output device names, plus AFC, `S/N @ 2500Hz`, baud-rate plot, and constellation diagram labels.	These are UI and diagnostic clues only. They do not reveal sample format, rates, modulation order, synchronization, coding, or demodulator design.
Static clue	Strings include `CQFRAME`, `LISTEN CQ`, `LISTEN ON/OFF`, `CHAT ON/OFF`, `RETRIES`, `VERSION`, `BITRATE`, and status/error text.	Candidate application-interface messages. Direction, syntax, response, ordering, and state semantics remain unverified.
Static clue	Ghidra cross-references map `CQFRAME`, `LISTEN ON/OFF`, and `VERSION` to `FUN\_004cdf80`; `CONNECTED`, `DISCONNECTED`, `BUSY ON/OFF`, `BITRATE`, and `VERSION VARA` reference separate functions `FUN\_004d14e0`, `FUN\_004d1ac0`, `FUN\_004d2640`, `FUN\_004d27a0`, `FUN\_004d2da0`, and `FUN\_004d33b0`.	These are static string xrefs in the VB6 application. The decompiler still cancels on these candidates; no command parsing, output, or state transition has been confirmed.
Static clue	ChangeLog entries mention KISS TCP frame types, fragmentation, DCD, persistence, speed levels 1/2/4, `BITRATE`, `DRIVELEVEL`, and special callsign suffixes.	Useful test-case leads; validate each against v4.9 behavior and protocol captures before encoding it in the implementation.
Static clue	Payload includes `VARAHF500.dat`, `VARAHF2300.dat`, and `VARAHF2750.dat`.	Distinct mode-named data files exist. Their data structures and role are unknown; no binary reverse-engineering conclusion is recorded yet.

Class	Observation	Interpretation / limit
Pending	Some sampled Ghidra decompilation requests returned `decompile_failed:cancelled`, including candidate functions near `004c9d90`, `004ca190`, `004cdf80`, and `004d2da0`.	Cross-reference and disassembly observations remain useful navigation evidence, but function semantics are not considered confirmed from those failed decompilations.

4.4 Saved Recon bookmarks

The analysis program has saved bookmarks at `00448260` (command-port UI text), `004cdf80` (references to CQ/listen text), `004d2da0` (BITRATE text), `00448a54` (sound-device readiness error), `00449164` (CQFRAME), `00440164` (WinMM audio API strings), `00442d1c` (audio input/output device labels), `004412a0` (audio diagnostics labels), `00449244` (RETRIES), and `00449508` (connection/status strings). These bookmarks are navigation aids, not proof that each referenced function implements the apparent behavior.

5 Rust scaffold and cross-platform environment

The crate is under `rust/`, package name `vara-modem`, edition 2024, Rust minimum 1.85. It currently has no third-party dependencies. Module boundaries are:

- `api`: future application-facing command/status interface;
- `audio`: future audio-device integration;
- `modem`: future portable modem/link behavior;
- `platform`: operating-system integration.

The executable prints help and version only. The crate deliberately does not claim modem functionality.

Rust 1.99.1 stable, Cargo, rustfmt, Clippy, and rust-analyzer are installed through rustup. Target standard libraries were installed for these six platform/architecture pairs:

```
aarch64-apple-darwin
x86_64-apple-darwin
aarch64-pc-windows-msvc
x86_64-pc-windows-msvc
aarch64-unknown-linux-gnu
x86_64-unknown-linux-gnu
```

`cargo check` passed for each target. This is cross-target Rust compile validation only; it is not native Windows/Linux execution, linking of platform dependencies, packaging, or audio-device validation.

Commands used for the Rust checks:

```
cd rust
cargo fmt --check
cargo clippy --all-targets -- -D warnings
```

```
cargo test
cargo check --target aarch64-apple-darwin
cargo check --target x86_64-apple-darwin
cargo check --target aarch64-pc-windows-msvc
cargo check --target x86_64-pc-windows-msvc
cargo check --target aarch64-unknown-linux-gnu
cargo check --target x86_64-unknown-linux-gnu
```

6 Workspace organization

Reference assets now live under `reference/vara/` and `reference/ghidra/`. The extracted v4.9 application payload is kept in `reference/vara/extracted-v4.9.0/app/`. Rust source and build instructions are under `rust/`; workspace overview is `README.md`. The live Ghidra project, VS Code MCP config, and bridge virtual environment remain at their existing paths. The current workspace is not a Git repository, so the notes do not identify a commit or claim source control tracking.

7 Implementation roadmap (proposed)

This roadmap is a plan, not a claim that the listed behavior has been reverse-engineered or implemented. Work is organized around measurable interoperability gates. In particular, waveform and decoder compatibility should be established before substantial investment in ARQ, host integration, or a graphical interface. The current crate remains a scaffold; crate names below are logical boundaries and do not require creating every crate up front.

7.1 Planning targets and architecture

The proposed product is an independently implemented Rust modem that interoperates with an unmodified VARA HF installation. The existing project scope is macOS, Windows, and Linux on ARM64 and x86_64; native build, audio, and device validation are separate acceptance requirements for each supported platform. A modular Rust workspace should separate:

- applications and diagnostics (CLI, analyzer, and, later, GUI);
- host command/data interfaces and link/ARQ state;
- physical-layer synchronization, modulation, and waveform processing;
- coding, framing, and reusable DSP primitives; and
- platform adapters for audio, PTT, CAT, serial, and configuration.

Start with the existing crate and split components when stable interfaces and tests justify the boundary. Candidate libraries include `rustfft` and `num-complex` for analysis, `hound` for WAV fixtures, `cpal` for audio, `tokio` for asynchronous networking, `serde`/`toml` for configuration, and `tracing` for diagnostics. These are options to evaluate, not dependencies already selected. Keep the modem native Rust; Python may be used for optional research tooling.

The proposal's approximately 2.4 kHz bandwidth, 48 kHz audio, OFDM, eleven adaptive levels, FSK ACK signaling, turbo coding, and TCP compatibility are candidate requirements to investigate. None is established by the current evidence. Record measured values and their supporting

fixtures before treating them as implementation requirements. SDR support is a possible later extension, not a first-release gate.

7.2 Phases and exit gates

The estimates below are rough engineering effort, not calendar commitments; parallelism, reference access, and undocumented behavior may change them. Each phase exit requires the stated evidence and regression tests, not merely compiling code.

Phase 0 — Research and reference environment (1–2 weeks). Establish authorized reference access, available specifications, a controlled audio loopback/capture setup, and a clean-room workflow. Record idle and representative non-radiating audio observations; maintain a corpus with provenance and known limitations. Deliver a research record, initial waveform corpus, and compatibility requirements with unknowns called out. Exit when a reference observation can be reproduced and its test conditions documented. Dynamic transmission work remains limited to controlled, non-radiating setups.

Phase 1 — Rust foundation and waveform analyzer (2–4 weeks). Retain the current scaffold while establishing shared error/logging conventions, WAV sample handling, spectral analysis, and machine-readable analysis output. Add visualization only where it helps inspect fixtures. Deliver an analyzer and deterministic tests over the captured corpus. Exit when it consistently detects and measures features demonstrated by the reference recordings; do not encode guessed waveform constants as facts.

Phase 2 — Synchronization and ACK receiver (2–4 weeks). Using evidence from Phase 0/1, characterize signal detection, frequency offset, timing, ACK signaling, symbol mapping, and session-dependent encoding. Implement a receiver only for behaviors supported by validated captures. Deliver repeatable ACK regression fixtures and command classifications where the evidence supports them. Exit when validated captures decode reliably across the documented cases; revise the phase if the proposed ACK model is incorrect.

Phase 3 — DATA physical-layer receiver (4–8 weeks). Characterize synchronization, carrier mapping, pilots, tracking, channel estimation, equalization, and demodulation from reference captures. Preserve soft information when required by the measured coding chain. Deliver a symbol-level receiver with per-capture diagnostics. Exit when it demodulates the validated reference cases across every confirmed transmission mode or level; the proposal's eleven-level count is not presumed.

Phase 4 — FEC and DATA decoding (4–8 weeks). Determine the actual encoder, interleaver, puncturing, frame-control, and integrity-check behavior before implementing it. Build encoder/decoder primitives and fixtures from independently observed inputs and outputs. Deliver complete frame decoding with payload and integrity validation. Exit when captured payloads are recovered and checks pass for the validated cases.

Phase 5 — Compatible transmitter (4–6 weeks). Implement the measured bit/frame construction, coding, modulation, pilots, and waveform generation. Add sample-level regression checks and validate against an unmodified reference receiver in a controlled, non-radiating setup. Exit only when the reference receiver accepts OpenVARA transmissions; visual similarity alone is insufficient.

Phase 6 — ARQ and session management (4–8 weeks). After physical-layer compatibility, implement the evidenced negotiation, roles, sequencing, ACK/NACK, retransmission, turnaround, adaptive behavior, timeouts, and PTT timing. Deliver a tested link state machine and transfer fixtures. Exit when bidirectional transfers complete reliably with an unmodified reference station under controlled test conditions.

Phase 7 — Host interface and platform integration (3–5 weeks). Confirm command and data socket semantics before implementing a compatible host interface. Add audio backends, configuration, and any evidenced CAT, serial PTT, or VOX integration. Deliver platform builds and host-application tests. Exit when supported host applications work without modification and the audio/PTT paths pass native validation on each claimed platform.

Phase 8 — Hardening and initial release (3–6 weeks). Profile and optimize only after correctness; add channel simulations, multipath/Doppler cases where applicable, extended interoperability tests, CI for formatting, linting, tests, and cross-platform builds, plus user and developer documentation. Review licensing and trademarks before distribution. The proposed 0.1 release gate is reproducible builds and demonstrated interoperability, not parity with VARA's maximum throughput or weak-signal performance.

7.3 Cross-cutting standards and acceptance

Use stable Rust and the existing edition/toolchain policy. Require formatting, Clippy, unit and integration tests, and cross-target compilation in CI; run native audio and interoperability tests separately because compilation does not demonstrate device behavior. Prefer deterministic DSP and explicit error propagation. Avoid heap allocation in real-time callbacks where feasible, and require documented justification and review for unsafe code in DSP/protocol paths. Benchmark and optimize against measured workloads.

Release acceptance should include, at minimum: validated synchronization; decoded frames with integrity checks; reference acceptance of transmitted frames; reference decoding by the OpenVARA receiver; bidirectional ARQ transfers; verified host-interface behavior; correct measured turnaround and PTT timing; native platform/audio checks; and extended regression testing. Only claim the modes, platforms, and conditions actually tested.

7.4 Initial execution order

The first implementation milestone should be a modest *Foundation 0.1*: make the current Rust scaffold a testable workspace, add WAV fixture support and basic spectrum analysis, and establish CI for the supported targets. In parallel, unblock the authorized reference runtime and collect reproducible captures. Then choose the next narrow receiver task based on measured evidence. Defer the GUI, broad host compatibility, full ARQ, and speculative protocol implementation until their prerequisite gates are satisfied.

8 Signal data acquisition and corpus (proposed)

8.1 Purpose and evidence policy

Build a traceable corpus of authentic VARA HF audio to support signal detection, synchronization, OFDM/ACK analysis, modulation and coding research, frame decoding, ARQ characterization,

and interoperability tests. Compatibility with unmodified VARA HF is the target: generic OFDM, simulated signals, and VARA FM recordings are not substitutes for VARA HF reference waveforms.

Treat every conclusion as evidence with provenance. Use the labels `DOCUMENTED` (explicitly stated in a technical reference), `MEASURED` (measured from a recording), `VALIDATED` (confirmed by repeatable decoding or interoperability tests), `HYPOTHESIS` (plausible but unconfirmed), or `UNKNOWN` (insufficient evidence). Do not promote a hypothesis based only on visual resemblance or an isolated capture. Update roadmap assumptions when new evidence changes scope, ordering, or estimated complexity.

8.2 Candidate data sources

1. **Controlled virtual-audio sessions (preferred starting point).** The user-provided guide points to the N4EAC VARA Modem Tester, a Windows test harness described as connecting two installed VARA HF instances over virtual audio cables, exchanging payloads, and recording audio and session metadata. Its repository documentation should be checked before use. The guide lists Windows, legitimate VARA installations, independent virtual-audio routes and ports, Python 3.12, and sufficient storage as prerequisites. The harness does not supply VARA, virtual-audio drivers, or a reference corpus. It is a proposed capture tool, not evidence that every rate or frame type has been exercised.
2. **Actual HF receive recordings.** These add real-channel conditions not represented by clean virtual audio. Record radio receive audio and, where available and authorized, the modem-to-radio transmit audio path or cooperating station's capture. Begin receive-only or use a properly attenuated, controlled setup. Any over-the-air transmissions must follow the operator's licensing, authorization, and applicable operating procedures. Obtain permission before retaining or sharing recordings; do not publish private message contents without authorization.
3. **Public references and samples.** The Signal Identification Wiki VARA HF page is background and a pointer, not confirmation that downloadable raw audio is available. The VARA HF Specification Revision 2.0 may inform documented expectations, but the guide dates it to April 2018; treat it as a historical specification, not a complete description of current behavior. The Freesound VARA FM recording may exercise WAV ingestion and analysis tooling only. It is VARA FM, not a VARA HF protocol fixture.
4. **Official VARA software.** The guide points to the Winlink VARA download page and downloads.winlink.org as sources for legitimate reference software. Software installation alone does not replace recorded, labeled signal data.

8.3 Recording preservation and metadata

Prefer complete, unprocessed WAV recordings: 48 kHz, mono where possible, uncompressed PCM at 16- or 24-bit depth, covering a complete transmission or session. These are capture preferences, not established VARA requirements. If the equipment produces another format or rate, preserve that original. Analysis copies may be converted, but must be traceable to the original. Never normalize, apply AGC, trim silence, denoise, filter, compress to MP3, or resample the only copy of a reference recording.

For every recording, retain a JSON sidecar and SHA-256 hash of the original. The sidecar should identify capture ID, mode, source type, file, audio properties, bandwidth when known, direction,

DATA level when independently established, session, expected payload references, contact status, payload verification, SNR/frequency offset when measured, UTC capture time, source, license/permission status, and notes. Unknown values must be JSON `null`; never invent a DATA level, SNR, or frequency offset. A metadata example is a schema illustration only, not a record of acquired data.

```
{
  "capture_id": "vara_hf_bw2300_session_0001",
  "mode": "VARA_HF",
  "source_type": "controlled_virtual_audio",
  "file": "station_1.wav",
  "sample_rate": 48000,
  "channels": 1,
  "bit_depth": 16,
  "encoding": "PCM",
  "bandwidth": "BW2300",
  "direction": "station_1_transmit",
  "data_level": null,
  "session_id": "session_0001",
  "payload_file": "payload.bin",
  "expected_payload_file": "expected_payload.bin",
  "contact_status": "complete",
  "payload_verified": true,
  "snr_db": null,
  "frequency_offset_hz": null,
  "capture_date_utc": "YYYY-MM-DDTHH:MM:SSZ",
  "source": "N4EAC VARA Modem Tester",
  "license": "recording_owner_permission_required",
  "sha256": "computed hash of the original WAV",
  "notes": "Illustrative schema only; not an acquired capture"
}
```

8.4 Coverage and capture campaigns

Maintain a coverage report that distinguishes available, missing, and insufficiently validated cases. Initial categories from the guide are: connection handshake; START; ACK1/ACK2/ACK3; NACK; BREAK; REQ; QRT; DATA levels 1–4, 5–8, and 9–11; both directions; complete sessions; retransmission/failure; weak signal; frequency offset; and fading/multipath. The ACK names and eleven-level grouping are drawn from the supplied guide's published-specification summary and are not independently validated here. Bandwidth and data level are distinct: never infer a DATA level from BW500, BW2300, or BW2750, or assign a level without reliable metadata or independent signal analysis.

Use staged campaigns:

1. **Clean baseline:** use two VARA HF instances over separately routed virtual audio; begin at BW2300. Exchange a short known text payload (for example `OPENVARA TEST 001`), a longer repeatable text payload, and a known binary payload (the guide's example is `00 7F 80 FF 55 AA 00 FF`). Record both directions and compare transmitted and received payloads exactly. First establish one successful session before expanding the run.
2. **Bandwidth coverage:** repeat both directions at BW500, BW2300, and BW2750. A session's

observed rate report is useful metadata, but does not establish a frame-level modulation assignment.

3. **Adaptive rates:** vary channel conditions where possible and seek low-, intermediate-, and high-rate examples, rate increases and decreases, and retransmissions. Do not assume one session covers all eleven published levels; verify each assignment independently.
4. **Real channel:** for permitted HF captures, note frequency/band, UTC time, receiving-station configuration, signal strength or SNR if available, interference, frequency offset if measurable, and decode outcome. Obtain permission and protect private content.
5. **Failure and recovery:** collect poor synchronization, weak signals, failed DATA, NACK/retransmission, failed-ACK recovery, termination, and interrupted sessions under controlled, authorized conditions.

8.5 Storage, analysis, and corpus admission

The proposed repository layout is a dedicated `data/` tree with `public/vara_hf` and `public/vara_fm`, `generated/clean`, `generated/bandwidth`, `generated/data_levels`, `generated/acknowledgments`, `generated/sessions`, field-capture folders for clean/fading/ interference/weak-signal cases, `derived/` folders for plots and measurements, and `reports/` for coverage and analyses. This is a proposal, not a statement that these directories currently exist. Keep large or restricted files outside the public source tree where appropriate; retain a manifest and approved-storage references. Do not commit recordings without permission and redistribution rights.

For each candidate recording, validate the WAV header and calculate duration, RMS, peak, and clipping percentage. Produce waveform and spectrogram views; measure occupied bandwidth and burst start/end, and estimate symbol timing, cyclic-prefix behavior, carrier spacing, frequency offset, and candidate pilots only when the data supports those measurements. Compare with published references, record confidence and evidence, and save a machine-readable analysis report. Map results to implementation areas: waveform/spectrum to DSP; burst detection and symbol timing to synchronization; carrier mapping to OFDM; ACK tones to FSK; modulation mapping to codec; soft decisions to FEC; DATA/ACK framing to link; retransmission/changeover to ARQ; and end-to-end traces to link and ARQ.

A recording enters the validated reference corpus only when it is readable, the original is preserved, format and sample properties are identified, actual modem activity is supported by evidence, source/capture method and permission status are documented, a sidecar and hash exist, category assignment is justified, and any expected payload has been independently verified. Quarantine incomplete or uncertain files; do not use them as golden fixtures. Existence of a WAV file is not proof of a successful capture.

8.6 Initial acquisition sprint and gate

The proposed first sprint favors a small, well-labeled corpus over many unverified files. Target one clean session each at BW2300, BW500, and BW2750; three station-1 and three station-2 WAV recordings; three known text payloads and one known binary payload; a sidecar for every recording; a complete SHA-256 manifest; waveform plots and spectrograms for every recording; and one coverage report. These are targets, not current assets or completed measurements.

This corpus can start WAV ingestion, waveform analysis, signal detection, and early synchronization

work; it is not sufficient to claim full compatibility. The immediate data milestone is a reproducible, validated collection of original VARA HF WAVs with known payloads, metadata, hashes, and analysis reports. Do not implement the complete modem solely from a published specification. The recommended order is to establish the capture harness and corpus/metadata first, analyze every accepted recording with repeatable tools, and then add permitted real-channel captures. Reassess coverage because a capture tool is not proof that all rate levels or control frames were tested.

9 Current readiness

Ready / observed	Still pending
Ghidra 12.1.4 plus matching extension; live plugin health endpoint; actual VARA v4.9 executable imported; initial Recon bookmarks; extracted reference payload preserved; Rust scaffold; six target checks; UTM and CrystalFetch installed; Windows installer ISO downloaded and located; UTM MCP configured; the user reports the `Windows` VM installation and ARM64 guest are configured; both UTM guests are currently stopped; the Windows guest has not provided usable network/runtime access; user states written authorization covers the analysis scope; UTM share `~/Downloads/utm-share/` is registered and contains the extracted v4.9.0 payload; command-related strings and additional function xrefs have been mapped in Ghidra.	Fedora install/first boot; guest network/runtime visibility; confirmed command behavior and controlled TCP/audio captures; confirmed command protocol; confirmed DSP/on-air behavior; native OS/audio tests; any working Rust modem features. The Windows ISO is present in Downloads.

10 Chronicle maintenance

Update this log during the work, not only when milestones are complete. Add dated entries when the analysis scope changes, new evidence is collected, code or tooling is changed, a validation command is run, or a blocker is discovered or cleared. Label deductions as hypotheses until independently verified. Refresh the presentation when a milestone, demo capability, or material readiness claim changes. Keep generated PDFs rebuilt from the LaTeX sources so the presentation and notes do not diverge.

11 Next work

1. The user reports written authorization covering the VARA research scope; preserve its boundaries and do not redistribute the software. The user reports v4.9.0 is installed in the Windows 11 ARM64 VM, but that guest is currently stopped and has not provided runtime access.

2. Continue read-only Ghidra work: follow the candidate command/status string references using assembly and xrefs. Treat each function mapping as a lead until decompilation or independent observations confirm behavior.
3. Keep protocol and DSP implementation deferred until command framing, direction, and on-air behavior have supporting evidence. Define acceptance criteria and deterministic fixtures from confirmed facts.
4. When a runtime environment becomes available, first capture idle, non-transmitting behavior and a read-only command such as VERSION; record exact bytes, socket identity, timing, and state transitions.
5. Revisit the separate Fedora/Wine fallback only after resolving why the previous Kickstart run exited without installing a system. Treat any Wine results as compatibility-specific until reproduced on Windows.

12 References

1. Supplied Ghidra beginner guide: local guide. Relevant subjects include program import, decompiler, function graphs, and cross-references.
2. VARA HF v4.9.0 local installer and extracted ChangeLog: `reference/vara/installers/` and `reference/vara/extracted-v4.9.0/app/ChangeLog.txt`.
3. VARA product overview: https://varamodem.com/.
4. UTM Windows guest guide: https://docs.getutm.app/guides/windows/.
5. Microsoft Windows 11 ARM64 download: https://www.microsoft.com/software-download/windows11arm64.